G2++ Model Due Diligence

Yes. Better format:

1. **No LaTeX escaping.**
2. **One function per block.**
3. **Each block is copy-pasteable MATLAB.**
4. **Header says where it goes: inside class or separate `.m` file.**

The methods below start at `getLambdaAndBy` and continue through calibration. They assume the upstream G2++ class already has the bucket, variance, bond-basket, and forward-measure methods we audited from your current class file. `% Gaussian Two-Fator Model aka G2++.txt`

Annex A.1 - `getLambdaAndBy`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function [lambda, By] = getLambdaAndBy(obj, strike, currMaturity, ulSwapSteps,
x_state, args)
    % Returns:
    %
    %     lambda_i(x) = c_i * A(T0,Ti) * exp(-Bx(T0,Ti)*x)
    %
    % and:
    %
    %     By_i = B(b,T0,Ti)

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        x_state (1,1) double = 0;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
    end

    instr = obj.getSwaptionInstrument( ...
        strike, currMaturity, ulSwapSteps, ...
        'direction', args.direction, ...
        'notional', args.notional);

    basket = obj.getSwaptionBondBasket(currMaturity, ulSwapSteps);

    coeffs = instr.coeffs(:);
    A0Ti = basket.A0Ti(:);
    Bx0Ti = basket.Bx0Ti(:);
    By0Ti = basket.By0Ti(:);

    lambda = coeffs .* A0Ti .* exp(-Bx0Ti .* x_state);
    By = By0Ti;
end
```

Annex A.2 - `getYBarValue`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function value = getYBarValue(obj, y, lambda, By)
    % Scalar equation for yBar:
    %
    %     f(y) = 1 - sum_i lambda_i(x) * exp(-By_i * y)

    arguments
        obj (1,1) GaussianTwoFactors;
        y (1,1) double;
        lambda (:,1) double;
        By (:,1) double;
    end

    value = 1 - sum(lambda .* exp(-By .* y));
end
```

Annex A.3 - `yBarSolverFromLambda`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function yBar = yBarSolverFromLambda(obj, lambda, By, args)
    % Solves for yBar(x):
    %
    %     1 = sum_i lambda_i(x) * exp(-By_i * yBar)

    arguments
        obj (1,1) GaussianTwoFactors;
        lambda (:,1) double;
        By (:,1) double;
        args.ySeed (1,1) double = 0;
        args.maxExpansion (1,1) double = 30;
        args.tolerance (1,1) double = 1e-10;
    end

    lambda = lambda(:);
    By = By(:);

    scalarF = @(y) obj.getYBarValue(y, lambda, By);

    yLow = args.ySeed - 0.01;
    yHigh = args.ySeed + 0.01;

    fLow = scalarF(yLow);
    fHigh = scalarF(yHigh);

    expansion = 0;

    while fLow * fHigh > 0 && expansion < args.maxExpansion
        width = yHigh - yLow;

        yLow = yLow - 2 * width;
        yHigh = yHigh + 2 * width;

        fLow = scalarF(yLow);
        fHigh = scalarF(yHigh);

        expansion = expansion + 1;
    end

    if fLow * fHigh > 0
        yBar = NaN;
        return;
    end

    yBar = fzero(scalarF, [yLow, yHigh]);

    residual = scalarF(yBar);
```

```
    if ~isfinite(yBar) || abs(residual) > args.tolerance
        yBar = NaN;
    end
end
```

Annex A.4 - `yBarSolver`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function yBar = yBarSolver(obj, strike, currMaturity, ulSwapSteps, x_state, arg
s)
    % Wrapper around yBarSolverFromLambda using instrument inputs.

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        x_state (1,1) double = 0;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
        args.ySeed (1,1) double = 0;
    end

    [lambda, By] = obj.getLambdaAndBy( ...
        strike, currMaturity, ulSwapSteps, x_state, ...
        'direction', args.direction, ...
        'notional', args.notional);

    yBar = obj.yBarSolverFromLambda(lambda, By, 'ySeed', args.ySeed);
end
```

Annex A.5 - `h1Function`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function h1 = h1Function(obj, yBar, muYCond, sigmaYCond)
    % Brigo h1:
    %
    %     h1(x) = (yBar(x) - E[Y|X=x]) / Std[Y|X=x]

    arguments
        obj (1,1) GaussianTwoFactors;
        yBar (1,1) double;
        muYCond (1,1) double;
        sigmaYCond (1,1) double {mustBeNonnegative};
    end

    if sigmaYCond <= 0
        error("sigmaYCond must be strictly positive.");
    end

    h1 = (yBar - muYCond) ./ sigmaYCond;
end
```

Annex A.6 - `h2Function`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function h2 = h2Function(obj, h1, By, sigmaYCond)
    % Brigo h2:
    %
    %     h2_i(x) = h1(x) + By_i * Std[Y|X=x]

    arguments
        obj (1,1) GaussianTwoFactors;
        h1 (1,1) double;
        By (:,1) double;
        sigmaYCond (1,1) double {mustBeNonnegative};
    end

    h2 = h1 + By(:) .* sigmaYCond;
end
```

Annex A.7 - `kappaFunction`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function kappa = kappaFunction(obj, By, muYCond, varYCond)
    % Brigo kappa_i(x):
    %
    %     kappa_i(x) = -By_i * muYCond + 0.5 * By_i^2 * varYCond

    arguments
        obj (1,1) GaussianTwoFactors;
        By (:,1) double;
        muYCond (1,1) double;
        varYCond (1,1) double {mustBeNonnegative};
    end

    By = By(:);

    kappa = -By .* muYCond + 0.5 .* (By.^2) .* varYCond;
end
```

Annex A.8 - `getStateGaussianAtExpiry`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function output = getStateGaussianAtExpiry(obj, currMaturity)
    % Computes the Gaussian law of (x(T0), y(T0)) under the T0-forward measure.

    arguments
        obj (1,1) GaussianTwoFactors;
        currMaturity (1,1) double;
    end

    T0 = currMaturity;

    muX = obj.muTForward(0, T0, ...
        'process', 'x_process', ...
        'tMaturity', T0);

    muY = obj.muTForward(0, T0, ...
        'process', 'y_process', ...
        'tMaturity', T0);

    varX = obj.varTForward(0, T0, 'process', 'x_process');
    varY = obj.varTForward(0, T0, 'process', 'y_process');
    covXY = obj.covTForward(0, T0);

    covMatrix = [varX, covXY; covXY, varY];

    output.muX = muX;
    output.muY = muY;
    output.varX = varX;
    output.varY = varY;
    output.covXY = covXY;
    output.covMatrix = covMatrix;
end
```

Annex A.9 - `getConditionalYMoments`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function output = getConditionalYMoments(obj, xVal, gaussianState)
    % Computes conditional moments of Y given X = xVal.

    arguments
        obj (1,1) GaussianTwoFactors;
        xVal (1,1) double;
        gaussianState (1,1) struct;
    end

    varX = gaussianState.varX;
    varY = gaussianState.varY;
    covXY = gaussianState.covXY;

    if varX <= 0
        error("varX must be strictly positive for conditional Gaussian formul
a.");
    end

    condMeanY = gaussianState.muY + (covXY / varX) * (xVal - gaussianState.mu
X);
    condVarY = varY - (covXY^2 / varX);

    condVarY = max(condVarY, 0);

    output.meanY = condMeanY;
    output.varY = condVarY;
    output.stdY = sqrt(condVarY);
end
```

Annex A.10 - `brigoSwaptionIntegrand`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function output = brigoSwaptionIntegrand(obj, z, omega, coeffs, A0Ti, Bx, By, g
aussianState)
    % Brigo G2++ swaption integrand using standardized normal variable z.
    %
    % x = muX + sigmaX * z
    %
    % Integrand:
    %
    % omega * [
    %     Phi(-omega*h1)
    %     - sum_i lambda_i(x) * exp(kappa_i(x)) * Phi(-omega*h2_i)
    % ] * phi(z)

    arguments
        obj (1,1) GaussianTwoFactors;
        z double;
        omega (1,1) double {mustBeMember(omega, [-1, 1])};
        coeffs (:,1) double;
        A0Ti (:,1) double;
        Bx (:,1) double;
        By (:,1) double;
        gaussianState (1,1) struct;
    end

    z = z(:).';
    output = zeros(size(z));

    muX = gaussianState.muX;
    varX = gaussianState.varX;

    sigmaX = sqrt(varX);

    if sigmaX <= 0
        error("sigmaX must be strictly positive.");
    end

    coeffs = coeffs(:);
    A0Ti = A0Ti(:);
    Bx = Bx(:);
    By = By(:);

    for j = 1:numel(z)

        xVal = muX + sigmaX * z(j);

        lambda = coeffs .* A0Ti .* exp(-Bx .* xVal);

        yBar = obj.yBarSolverFromLambda(lambda, By);
```

```
        if ~isfinite(yBar)
            output(j) = NaN;
            continue;
        end

        cond = obj.getConditionalYMoments(xVal, gaussianState);

        muYCond = cond.meanY;
        varYCond = cond.varY;
        sigmaYCond = cond.stdY;

        if sigmaYCond <= 0
            basketVal = sum(lambda .* exp(-By .* muYCond));

            if omega == 1
                condPayoff = max(1 - basketVal, 0);
            else
                condPayoff = max(basketVal - 1, 0);
            end

            output(j) = condPayoff * normpdf(z(j));
            continue;
        end

        h1 = obj.h1Function(yBar, muYCond, sigmaYCond);
        h2 = obj.h2Function(h1, By, sigmaYCond);
        kappa = obj.kappaFunction(By, muYCond, varYCond);

        bracket = normcdf(-omega .* h1) ...
                - sum(lambda .* exp(kappa) .* normcdf(-omega .* h2));

        output(j) = omega .* bracket .* normpdf(z(j));
    end
end
```

Annex A.11 - `priceSwaptionG2PP`

Location: inside `GaussianTwoFactors.m`, inside `methods`.

```
function price = priceSwaptionG2PP(obj, strike, currMaturity, ulSwapSteps, arg
s)
    % Prices a European swaption under G2++ using Brigo's explicit
    % one-dimensional integral.
    %
    % Convention:
    %   omega =  1 for payer
    %   omega = -1 for receiver

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
        args.integrationLower (1,1) double = -8;
        args.integrationUpper (1,1) double = 8;
        args.absTol (1,1) double = 1e-10;
        args.relTol (1,1) double = 1e-8;
    end

    if strcmpi(args.direction, 'PAY')
        omega = 1;
    else
        omega = -1;
    end

    instr = obj.getSwaptionInstrument( ...
        strike, currMaturity, ulSwapSteps, ...
        'direction', args.direction, ...
        'notional', args.notional);

    basket = obj.getSwaptionBondBasket(instr.expiry, instr.paymentDates);

    gaussianState = obj.getStateGaussianAtExpiry(instr.expiry);

    coeffs = instr.coeffs(:);
    A0Ti = basket.A0Ti(:);
    Bx = basket.Bx0Ti(:);
    By = basket.By0Ti(:);

    integrand = @(z) obj.brigoSwaptionIntegrand( ...
        z, omega, coeffs, A0Ti, Bx, By, gaussianState);

    expectation = integral( ...
        integrand, ...
```

```
            args.integrationLower, ...
            args.integrationUpper, ...
            'ArrayValued', true, ...
            'AbsTol', args.absTol, ...
            'RelTol', args.relTol);

        price = instr.notional .* basket.P0T0 .* expectation;

        if ~isfinite(price)
            price = NaN;
        end
    end
```

Annex B.1 - `buildSwaptionCalibrationSet.m`

Location: separate file named `buildSwaptionCalibrationSet.m`.

```
function calibSet = buildSwaptionCalibrationSet(expiries, tenors, quotes, args)
% Builds a flat calibration set from a swaption quote matrix.

    arguments
        expiries (1,:) double
        tenors (1,:) double
        quotes (:,:) double
        args.quoteType (1,:) char {mustBeMember(args.quoteType, {'price','blac
k','normal'})} = 'price'
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY'
        args.notional (1,1) double = 1.0
        args.strikeConvention (1,:) char {mustBeMember(args.strikeConvention,
{'ATM','fixed'})} = 'ATM'
        args.fixedStrike (1,1) double = NaN
        args.paymentFrequency (1,1) double {mustBePositive} = 1
        args.weight (:,:) double = []
    end

    nExp = numel(expiries);
    nTenor = numel(tenors);

    if size(quotes,1) ~= nExp || size(quotes,2) ~= nTenor
        error("quotes must have size numel(expiries) x numel(tenors).");
    end

    if isempty(args.weight)
        weights = ones(nExp, nTenor);
    else
        weights = args.weight;
        if any(size(weights) ~= size(quotes))
            error("weight must have the same size as quotes.");
        end
    end

    if strcmpi(args.strikeConvention, 'fixed') && ~isfinite(args.fixedStrike)
        error("fixedStrike must be provided when strikeConvention = 'fixed'.");
    end

    k = 0;
    calibSet = struct([]);

    for i = 1:nExp
        expiry = expiries(i);

        for j = 1:nTenor
            tenor = tenors(j);
            quote = quotes(i,j);
```

```
            if ~isfinite(quote)
                continue;
            end

            dt = 1 / args.paymentFrequency;
            nPayments = round(tenor * args.paymentFrequency);

            paymentDates = expiry + (1:nPayments) .* dt;

            k = k + 1;

            calibSet(k).expiry = expiry;
            calibSet(k).tenor = tenor;
            calibSet(k).paymentDates = paymentDates;
            calibSet(k).quote = quote;
            calibSet(k).quoteType = args.quoteType;
            calibSet(k).direction = args.direction;
            calibSet(k).notional = args.notional;
            calibSet(k).strikeConvention = args.strikeConvention;
            calibSet(k).fixedStrike = args.fixedStrike;
            calibSet(k).weight = weights(i,j);
        end
    end
end
```

Annex B.2 - `getSwaptionForwardAndAnnuityG2PP.m`

Location: separate file named `getSwaptionForwardAndAnnuityG2PP.m`.

```
function [forwardSwapRate, annuity, delta, P0T0, P0Ti] = ...
    getSwaptionForwardAndAnnuityG2PP(model, expiry, paymentDates)
% Computes single-curve forward swap rate and annuity.

    arguments
        model (1,1) GaussianTwoFactors
        expiry (1,1) double
        paymentDates (1,:) double
    end

    paymentDates = paymentDates(:);

    zc = model.getInitialMarketZC([expiry, paymentDates.']);
    zc = zc(:);

    P0T0 = zc(1);
    P0Ti = zc(2:end);

    delta = diff([expiry; paymentDates]);

    annuity = sum(delta .* P0Ti);

    forwardSwapRate = (P0T0 - P0Ti(end)) / annuity;
end
```

Annex B.3 - `swaptionPriceFromVol.m`

Location: separate file named `swaptionPriceFromVol.m`.

```
function price = swaptionPriceFromVol(F, K, T, annuity, vol, volType, directio
n, notional)
% Converts quoted normal or Black volatility into a swaption price.

    arguments
        F (1,1) double
        K (1,1) double
        T (1,1) double {mustBeNonnegative}
        annuity (1,1) double {mustBeNonnegative}
        vol (1,1) double {mustBeNonnegative}
        volType (1,:) char {mustBeMember(volType, {'normal','black'})}
        direction (1,:) char {mustBeMember(direction, {'PAY','REC'})} = 'PAY'
        notional (1,1) double = 1.0
    end

    omega = 1;
    if strcmpi(direction, 'REC')
        omega = -1;
    end

    if T == 0 || vol == 0
        intrinsic = max(omega * (F - K), 0);
        price = notional * annuity * intrinsic;
        return;
    end

    sqrtT = sqrt(T);

    switch lower(volType)

        case 'normal'
            std = vol * sqrtT;
            d = (F - K) / std;

            price = notional * annuity * ...
                (omega * (F - K) * normcdf(omega * d) + std * normpdf(d));

        case 'black'
            if F <= 0 || K <= 0
                error('Black volatility requires strictly positive F and K.');
            end

            std = vol * sqrtT;
            d1 = (log(F / K) + 0.5 * std^2) / std;
            d2 = d1 - std;

            price = notional * annuity * ...
                omega * (F * normcdf(omega * d1) - K * normcdf(omega * d2));
```

```
        end
    end
```

Annex B.4 - `swaptionImpliedVolFromPrice.m`

Location: separate file named `swaptionImpliedVolFromPrice.m`.

```
function impVol = swaptionImpliedVolFromPrice(price, F, K, T, annuity, volType,
direction, notional)
% Computes implied normal or Black volatility from a swaption price.

    arguments
        price (1,1) double {mustBeNonnegative}
        F (1,1) double
        K (1,1) double
        T (1,1) double {mustBeNonnegative}
        annuity (1,1) double {mustBeNonnegative}
        volType (1,:) char {mustBeMember(volType, {'normal','black'})}
        direction (1,:) char {mustBeMember(direction, {'PAY','REC'})} = 'PAY'
        notional (1,1) double = 1.0
    end

    omega = 1;
    if strcmpi(direction, 'REC')
        omega = -1;
    end

    intrinsic = notional * annuity * max(omega * (F - K), 0);

    if price < intrinsic - 1e-12
        impVol = NaN;
        return;
    end

    if abs(price - intrinsic) < 1e-14
        impVol = 0;
        return;
    end

    objFun = @(v) swaptionPriceFromVol( ...
        F, K, T, annuity, v, volType, direction, notional) - price;

    switch lower(volType)

        case 'normal'
            volLow = 1e-10;
            volHigh = 5.0;

        case 'black'
            if F <= 0 || K <= 0
                impVol = NaN;
                return;
            end
            volLow = 1e-10;
            volHigh = 5.0;
```

```
    end

    while objFun(volHigh) < 0
        volHigh = 2 * volHigh;

        if volHigh > 100
            impVol = NaN;
            return;
        end
    end

    impVol = fzero(objFun, [volLow, volHigh]);
end
```

Annex B.5 - `getCalibrationStrikeG2PP.m`

Location: separate file named `getCalibrationStrikeG2PP.m`.

```
function K = getCalibrationStrikeG2PP(model, instr)
% Returns fixed or ATM strike for one calibration instrument.

    arguments
        model (1,1) GaussianTwoFactors
        instr (1,1) struct
    end

    [F, ~] = getSwaptionForwardAndAnnuityG2PP( ...
        model, instr.expiry, instr.paymentDates);

    if strcmpi(instr.strikeConvention, 'ATM')
        K = F;
    else
        K = instr.fixedStrike;
    end
end
```

Annex B.6 - `swaptionMarketPriceFromQuote.m`

Location: separate file named `swaptionMarketPriceFromQuote.m`.

```
function marketPrice = swaptionMarketPriceFromQuote(model, instr)
% Converts a market quote into a market swaption price.

    arguments
        model (1,1) GaussianTwoFactors
        instr (1,1) struct
    end

    [F, A] = getSwaptionForwardAndAnnuityG2PP( ...
        model, instr.expiry, instr.paymentDates);

    if strcmpi(instr.strikeConvention, 'ATM')
        K = F;
    else
        K = instr.fixedStrike;
    end

    switch lower(instr.quoteType)

        case 'price'
            marketPrice = instr.notional * instr.quote;

        case {'normal','black'}
            marketPrice = swaptionPriceFromVol( ...
                F, K, instr.expiry, A, instr.quote, instr.quoteType, ...
                instr.direction, instr.notional);

        otherwise
            error("Unsupported quote type.");
    end
end
```

Annex B.7 - `g2ppInverseTransformBrigo5.m`

Location: separate file named `g2ppInverseTransformBrigo5.m`.

```
function thetaRaw = g2ppInverseTransformBrigo5(a, b, sigma, eta, rho)
% Maps admissible five-parameter G2++ parameters to unconstrained variables.

    arguments
        a (1,1) double {mustBePositive}
        b (1,1) double {mustBePositive}
        sigma (1,1) double {mustBePositive}
        eta (1,1) double {mustBePositive}
        rho (1,1) double
    end

    if abs(rho) >= 1
        error("rho must satisfy abs(rho) < 1.");
    end

    thetaRaw = [
        log(a);
        log(b);
        log(sigma);
        log(eta);
        atanh(rho)
    ];
end
```

Annex B.8 - `g2ppApplyTransformBrigo5.m`

Location: separate file named `g2ppApplyTransformBrigo5.m`.

```
function params = g2ppApplyTransformBrigo5(model, thetaRaw)
% Applies five-parameter constant-vol G2++ calibration to the model.
%
% thetaRaw = [log(a); log(b); log(sigma); log(eta); atanh(rho)]

    arguments
        model (1,1) GaussianTwoFactors
        thetaRaw (5,1) double
    end

    params.a = exp(thetaRaw(1));
    params.b = exp(thetaRaw(2));
    params.sigma = exp(thetaRaw(3));
    params.eta = exp(thetaRaw(4));
    params.rho = tanh(thetaRaw(5));

    model.meanReversion_x = params.a;
    model.meanReversion_y = params.b;
    model.correlation = params.rho;

    nSigma = numel(model.volatility_sigma);
    nEta = numel(model.volatility_eta);

    if nSigma == 0 || nEta == 0
        error("model.volatility_sigma and model.volatility_eta must be initiali
zed before calibration.");
    end

    model.volatility_sigma = params.sigma .* ones(1, nSigma);
    model.volatility_eta = params.eta .* ones(1, nEta);
end
```

Annex B.9 - `g2ppCalibrationResiduals.m`

Location: separate file named `g2ppCalibrationResiduals.m`.

```
function [residuals, diagnostics] = g2ppCalibrationResiduals(thetaRaw, model, c
alibSet, args)
% Residual vector for five-parameter constant-vol G2++ calibration.

    arguments
        thetaRaw (5,1) double
        model (1,1) GaussianTwoFactors
        calibSet (1,:) struct
        args.objectiveType (1,:) char {mustBeMember(args.objectiveType, {'pric
e','normalizedPrice','vol'})} = 'normalizedPrice'
        args.volTypeForDiagnostics (1,:) char {mustBeMember(args.volTypeForDiag
nostics, {'normal','black'})} = 'normal'
        args.badValuePenalty (1,1) double = 1e6
    end

    params = g2ppApplyTransformBrigo5(model, thetaRaw);

    n = numel(calibSet);
    residuals = zeros(n,1);

    diagnostics.modelPrice = NaN(n,1);
    diagnostics.marketPrice = NaN(n,1);
    diagnostics.modelVol = NaN(n,1);
    diagnostics.marketVol = NaN(n,1);
    diagnostics.strike = NaN(n,1);
    diagnostics.forward = NaN(n,1);
    diagnostics.annuity = NaN(n,1);
    diagnostics.expiry = NaN(n,1);
    diagnostics.tenor = NaN(n,1);
    diagnostics.params = params;

    for k = 1:n
        instr = calibSet(k);

        try
            K = getCalibrationStrikeG2PP(model, instr);

            [F, A] = getSwaptionForwardAndAnnuityG2PP( ...
                model, instr.expiry, instr.paymentDates);

            marketPrice = swaptionMarketPriceFromQuote(model, instr);

            modelPrice = model.priceSwaptionG2PP( ...
                K, instr.expiry, instr.paymentDates, ...
                'direction', instr.direction, ...
                'notional', instr.notional);

            diagnostics.modelPrice(k) = modelPrice;
```

```
            diagnostics.marketPrice(k) = marketPrice;
            diagnostics.strike(k) = K;
            diagnostics.forward(k) = F;
            diagnostics.annuity(k) = A;
            diagnostics.expiry(k) = instr.expiry;

            if isfield(instr, 'tenor')
                diagnostics.tenor(k) = instr.tenor;
            end

            if isfield(instr, 'quote') && any(strcmpi(instr.quoteType, {'norma
l','black'}))
                diagnostics.marketVol(k) = instr.quote;
            end

            switch args.objectiveType

                case 'price'
                    residuals(k) = sqrt(instr.weight) * (modelPrice - marketPri
ce);

                case 'normalizedPrice'
                    scale = max(instr.notional * A, 1e-12);
                    residuals(k) = sqrt(instr.weight) * (modelPrice - marketPri
ce) / scale;

                case 'vol'
                    modelVol = swaptionImpliedVolFromPrice( ...
                        modelPrice, F, K, instr.expiry, A, ...
                        args.volTypeForDiagnostics, instr.direction, instr.noti
onal);

                    diagnostics.modelVol(k) = modelVol;

                    if isfield(instr, 'quote') && strcmpi(instr.quoteType, arg
s.volTypeForDiagnostics)
                        marketVol = instr.quote;
                    else
                        marketVol = swaptionImpliedVolFromPrice( ...
                            marketPrice, F, K, instr.expiry, A, ...
                            args.volTypeForDiagnostics, instr.direction, instr.
notional);
                    end

                    diagnostics.marketVol(k) = marketVol;

                    if ~isfinite(modelVol) || ~isfinite(marketVol)
                        residuals(k) = args.badValuePenalty;
                    else
```

```
                        residuals(k) = sqrt(instr.weight) * (modelVol - marketV
ol);
                    end
            end

            if ~isfinite(residuals(k))
                residuals(k) = args.badValuePenalty;
            end

        catch
            residuals(k) = args.badValuePenalty;
        end
    end
end
```

Annex B.10 - `calibrateG2PPToSwaptionSet.m`

Location: separate file named `calibrateG2PPToSwaptionSet.m`.

```
function result = calibrateG2PPToSwaptionSet(model, calibSet, theta0Raw, args)
% Calibrates five-parameter constant-vol G2++ model to a swaption set.

    arguments
        model (1,1) GaussianTwoFactors
        calibSet (1,:) struct
        theta0Raw (5,1) double
        args.objectiveType (1,:) char {mustBeMember(args.objectiveType, {'pric
e','normalizedPrice','vol'})} = 'normalizedPrice'
        args.volTypeForDiagnostics (1,:) char {mustBeMember(args.volTypeForDiag
nostics, {'normal','black'})} = 'normal'
        args.maxIterations (1,1) double = 200
        args.display (1,:) char = 'iter'
        args.useLsqnonlin (1,1) logical = true
    end

    residualFun = @(theta) g2ppCalibrationResiduals( ...
        theta, model, calibSet, ...
        'objectiveType', args.objectiveType, ...
        'volTypeForDiagnostics', args.volTypeForDiagnostics);

    if args.useLsqnonlin

        opts = optimoptions('lsqnonlin', ...
            'Display', args.display, ...
            'MaxIterations', args.maxIterations, ...
            'StepTolerance', 1e-8, ...
            'FunctionTolerance', 1e-8);

        [thetaStar, resnorm, residuals, exitflag, output] = ...
            lsqnonlin(residualFun, theta0Raw, [], [], opts);

    else

        objFun = @(theta) sum(residualFun(theta).^2);

        opts = optimset( ...
            'Display', args.display, ...
            'MaxIter', args.maxIterations);

        [thetaStar, resnorm, exitflag, output] = ...
            fminsearch(objFun, theta0Raw, opts);

        residuals = residualFun(thetaStar);
    end

    params = g2ppApplyTransformBrigo5(model, thetaStar);
```

```
        [finalResiduals, diagnostics] = g2ppCalibrationResiduals( ...
            thetaStar, model, calibSet, ...
            'objectiveType', args.objectiveType, ...
            'volTypeForDiagnostics', args.volTypeForDiagnostics);

        result.thetaRaw = thetaStar;
        result.params = params;
        result.resnorm = resnorm;
        result.residuals = residuals;
        result.finalResiduals = finalResiduals;
        result.exitflag = exitflag;
        result.output = output;
        result.diagnostics = diagnostics;

        result.rmse = sqrt(mean(finalResiduals.^2));
        result.maxAbsError = max(abs(finalResiduals));
    end
```

This is now copy-paste oriented: **Annex A goes into the class**, **Annex B goes into individual `.m` files**.

Powered by [ChatGPT Exporter](https://www.chatgptexporter.com) (https://www.chatgptexporter.com)